real-tr-p8-noglossary

An AgentMPI run analysis

Abstract

This is the glossary ablation of the eight-rank parallel book translation: the `allreduce` that lets the ranks agree on a shared rendering for every proper noun is removed, and with it the term-extraction phase that fed it, leaving each rank to translate its section of *A Scandal in Bohemia* with no shared terminology at all. It was built to price a collective — cheaper and faster without it, internally inconsistent as a result — and it measures the first half of that trade precisely and fails to find the second half at all. Removing the glossary cut wall time from 166.78 s to 94.55 s, input tokens from 43,448 to 13,845, messages from 56 to 28, agent calls from 24 to 16, and cost from $0.2599 to $0.1355; the decomposition below shows that only 8.74 s of the 72.23 s saved is the two `allreduce` invocations themselves, and 45.5 s is the downstream cost of carrying a 206-term glossary in every subsequent prompt. The consistency the glossary exists to buy did not move: the pre-registered terminology-consistency score is 1.000 here, and 1.000 in all three siblings, and three of the eight assembled sections are byte-identical to the baseline's. The takeaway is two-sided. In an agent system the dominant cost of a collective is not the collective but the size of the artefact it injects into every later prompt; and this ablation ladder cannot currently price the benefit, because its quality metric is at its ceiling in every arm including the ones that keep the machinery.

1 What this run is

property	value
run	`real-tr-p8-noglossary`
experiment	`translation`
campaign label	`real-tr`
declared world size	8
ranks appearing in the log	10
executors	`broker` × 8, `worker` × 26
events	204
wall time	94.55 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	6.60×	total busy time over wall time
parallel efficiency	82.5%	achieved parallelism per rank
idle fraction	17.5%	rank-seconds paid for and unused
peak ranks busy	8	of 8 in the world
serial fraction of active time	3.2%	time with exactly one rank busy
load imbalance	1.11×	slowest rank's busy time over the mean
coordination share	20.1%	wall time inside collectives
rank reattachments	24	fresh agent processes that took over a rank role
agent calls	16	invocations that reached a model
agent latency p50 / p95	43.02 / 50.52 s	per invocation
messages	28	20 eager, 8 rendezvous
tokens sent	16,246	8,161 deferred under rendezvous
tokens in / out	13,845 / 6,263	prompt and completion
cost	$0.1355	0.0216 $/kTok out

3 What the analysis notices

- [warning] `halo_exchange/?` reports 0 messages but the fabric logged 14: the collective's own accounting disagrees with the traffic it produced.
- [warning] Ranks 8, 14 appear in the log without participating; the declared world size is 8. Shared worker pools let a worker register against the wrong job, which inflates the apparent rank count.
- [ok] 24 rank reattachment(s) occurred, up to incarnation 4: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 3 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	84.90	89.8	2	2	4	4	6,823	0.0	finalized
1	75.13	79.5	2	2	3	3	405	0.0	finalized
2	70.98	75.1	2	2	4	4	1,682	0.0	finalized
3	73.64	78.0	2	2	3	3	351	0.0	finalized
4	86.57	91.6	2	2	5	5	4,410	0.0	finalized
5	81.29	86.0	2	2	3	3	398	0.0	finalized
6	68.47	72.4	2	2	4	4	1,797	0.0	finalized
7	83.18	88.0	2	2	2	2	380	0.0	finalized

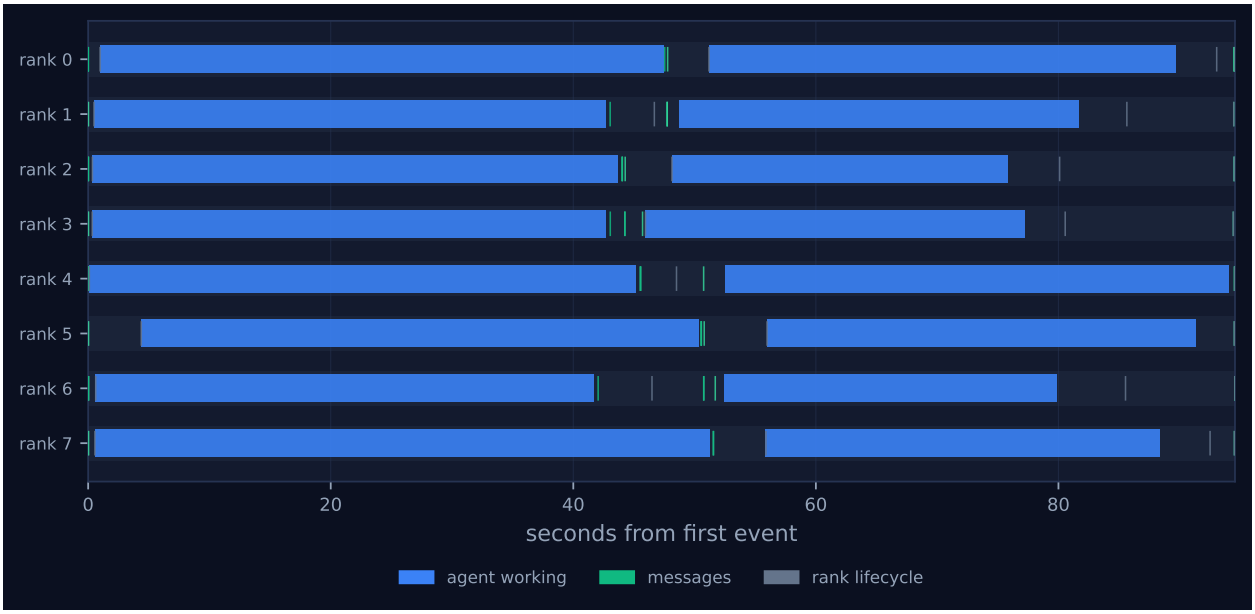

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

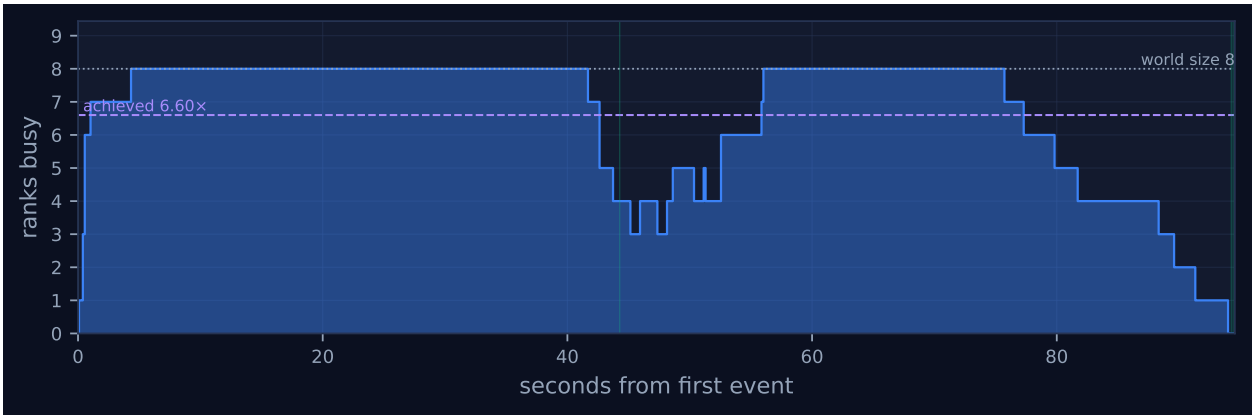

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

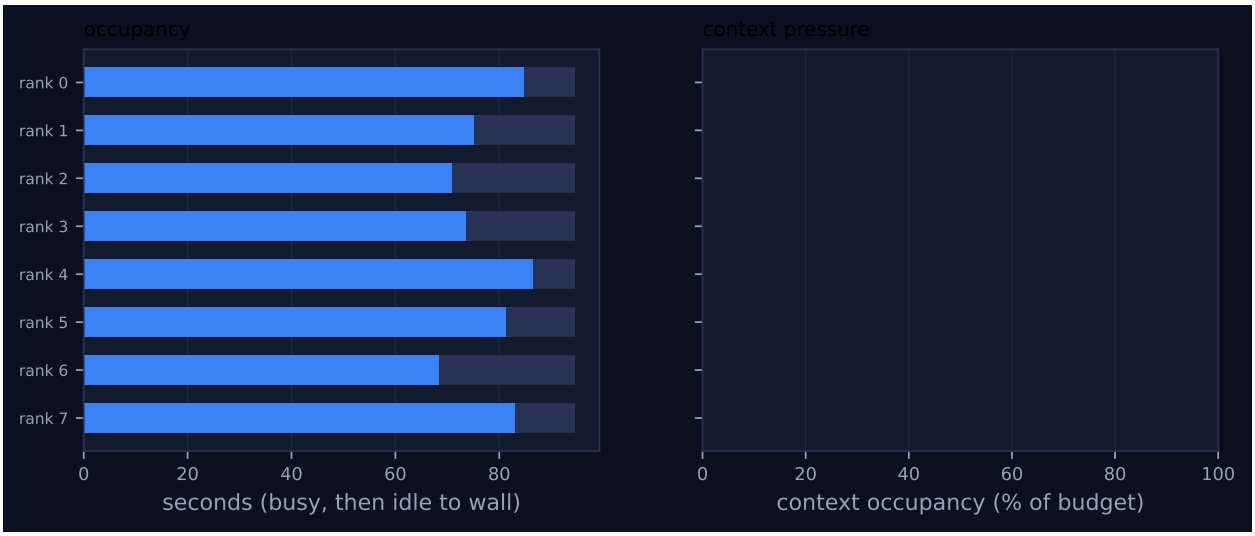

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
scatter	binomial	decompose	8	8	3	3	7	7	11,594	0.03	0.02	✓
halo_exchange	?	seams	8	8	0	–	14	–	0	0.00	7.43	–
barrier	central	pre-gather	8	8	0	2	0	0	0	18.69	0.24	✓
gather	binomial	collect	8	8	3	3	7	7	4,334	0.28	0.14	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 has the simplest shape any run in this family can have: eight lanes, each holding exactly two blue bars and nothing else. The first bar is the translation of one section, the second is the boundary revision, and the narrow band of green ticks between them at $t \approx 42$–$52\,\mathrm{s}$ is the halo exchange. There is no third bar and no left margin. The `scatterv` that distributes the book completes in 0.03 s of wall time having pushed 11,594 tokens down a binomial tree in three rounds, and every rank has enqueued its translation task within 40 ms of the first event. That empty left margin is the whole finding in visual form: in the baseline, the first 30.3 s of every lane is a term-sheet extraction call followed by two `allreduce` invocations, and here there is nothing there to draw.

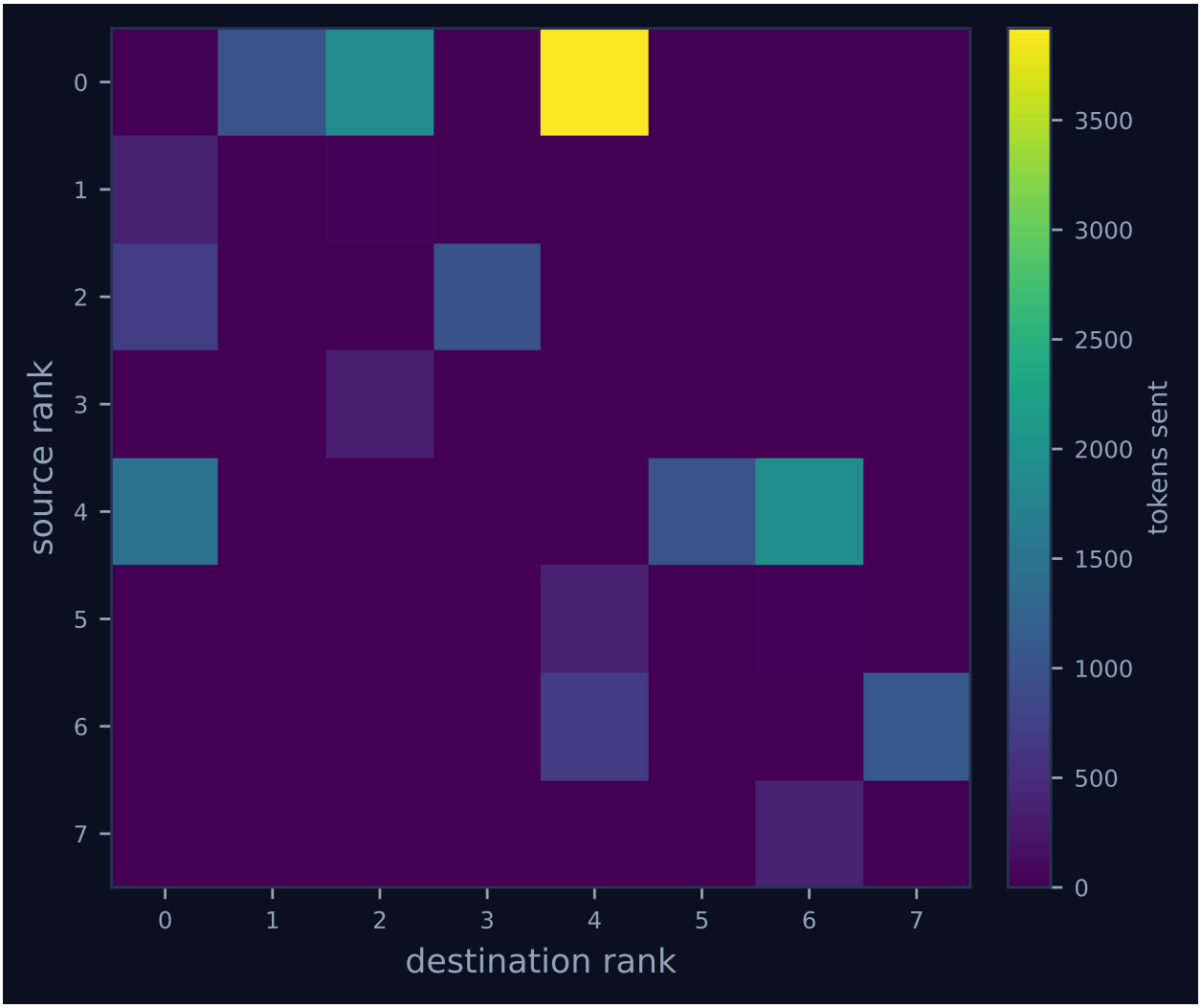

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

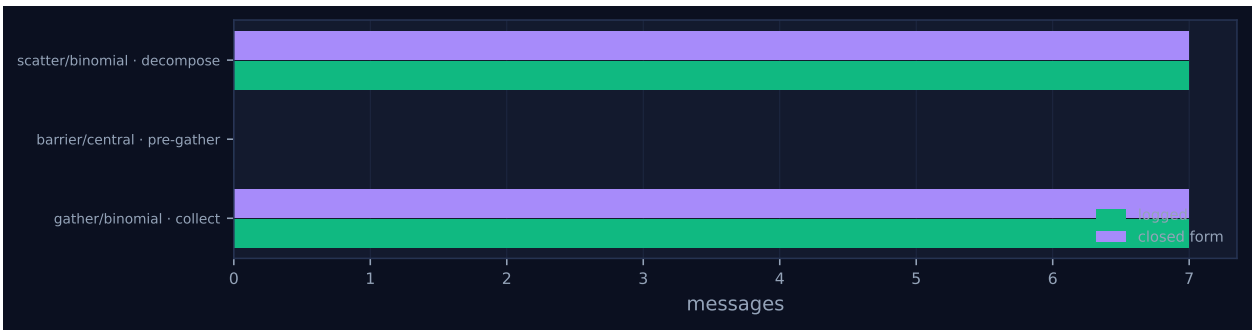

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

One lane starts late. Rank 5's task was enqueued at 0.04 s and no worker claimed it until 4.38 s, which is why its first bar is visibly offset in both Figure 1 and the viewer screenshot. This is broker queueing, not protocol: pairing every `broker.enqueue` with its `broker.claim` gives a median wait of 0.94 s, a maximum of 5.18 s (`revise:u5`, again rank 5), and 28.05 s summed over all sixteen tasks against 624 s of total busy time. The `max_claim_wait_s` of 0.0 in the headline table is a different quantity — it reports only `broker.expire` events, of which this run has none — so the queueing above had to be computed from the log directly.

Figure 2 shows why the run is not simply eight parallel translations. It holds a plateau at eight ranks busy until $t = 42$ s, drops to three between 44 and 56 s, recovers to eight, and then descends a staircase from 76 s to the end. The notch is the halo exchange, and it is a wavefront rather than a barrier: each rank must receive one paragraph from each Cartesian neighbour before it can start revising, so its start time is set by the slower of its two neighbours. Rank 6 finished translating first, at 42.04 s, and sat idle for 9.67 s waiting on a 5-token left boundary from rank 7, the last translator to finish at 51.55 s. Measuring each rank's gap between its `agent.call` for translation and its `coll.halo_exchange` gives 0.25, 4.70, 0.24, 2.66, 5.21, 0.24, 9.67 and 0.00 s for ranks 0 through 7 — 22.97 rank-seconds of idling bought by a ring that moved 363 tokens in total.

The staircase from 76 s is the pre-gather barrier draining. Rank 2 finished revising at 75.79 s and waited 18.69 s; rank 4 arrived last at 94.26 s and released everyone. The per-rank waits sum to 75.56 rank-seconds, matching the 75.59 s the collective summary attributes to the barrier, and the reported barrier wall of 18.69 s is exactly the longest single wait. The barrier ran under `PROCEED` and reported `absent` = [] on every rank: all eight arrived, nothing was declared missing, and the policy was never exercised. Release skew is 0.24 s and the `gather` that follows takes 0.28 s, so the entire collection phase costs a third of a second once the last rank shows up. The run's 20.1% coordination share is, almost entirely, one straggler.

Figure 4 and Figure 1 disagree about what matters, and both are right. The communication matrix is dominated by the scatter tree — the bright $0 \to 4$ cell is 3,919 tokens, with $0 \to 2$, $0 \to 1$, $4 \to 6$, $4 \to 5$, $2 \to 3$ and $6 \to 7$ below it — overlaid by the gather returning along the same edges. The ring that produces the middle notch of the timeline is invisible: its fourteen messages carry 363 tokens, 2.2% of the 16,246 sent. The halo is nearly free in bytes and expensive only in synchronisation, which is precisely the property a communication-volume figure cannot show.

Two things are absent from the figures and worth naming. The right-hand panel of Figure 3 is empty: every one of the 28 receives carries `admitted` = `false`, context high-water is 0 on all eight ranks, and nothing about context pressure can be read from this run. And the viewer's lane list runs to ten, with ranks 8 and 14 drawn as two lifecycle ticks and no work at all.

7 Interpretation

What is true by construction

The harness's `-no-glossary` flag does more than skip the `allreduce`. In the translation harness the term-extraction phase sits inside the same `if cfg.glossary:` guard, so this arm has no term sheets to withhold from each other: the log contains no `terms:` agent call, `n_local_terms` is 0 on all eight ranks, the extract phase is 0.0 s, and every rank records the same `glossary_digest` `44136fa355b3` — the digest of the empty dictionary. The binding-glossary block of the translation prompt reads `(none provided)`. The configuration is therefore best described as *no terminology*

machinery, not *local terminology without agreement*, and the comparison against the baseline prices the whole phase rather than the collective alone. The decomposition below separates the two.

Four collectives survive, and what they have in common is instructive: `scatterv` to distribute the sections, `halo_exchange` on a 1-D Cartesian ring to reconcile the seams, a `barrier` to contain stragglers, and a rendezvous `gather` to collect the result. Three of those exist because the work was decomposed at all — they would be needed to run any partitioned task over eight ranks — and only the halo exists because of what the task *means*. Removing the glossary removed the one collective whose purpose was to make the agents agree with each other about content rather than about control. That is the clean way to state what this arm subtracts.

What the removal saved

Against the baseline `real-tr-p8-full`, this run cuts events from 347 to 204, wall time from 166.78 s to 94.55 s (1.76× faster), agent calls from 24 to 16, messages from 56 to 28, tokens sent from 31,603 to 16,246, input tokens from 43,448 to 13,845, output tokens from 8,636 to 6,263, and cost from $0.2599 to $0.1355. Removing a collective made the run cheaper and faster, as it must; the interesting question is where the saving actually came from, and neither headline answer is right.

The input-token saving is exactly 29,603 tokens, and it decomposes without residue. Summing `prompt_tokens` over the `agent.call` events by phase: the eight term-sheet prompts that no longer exist account for 9,127; the translation prompts fell from a mean of 2,484 tokens to 1,161, saving 10,582; the revision prompts fell from 1,806 to 570, saving 9,894. Two thirds of the input-token cost of having a glossary is not the act of agreeing on one. It is the 206-term artefact being reprinted into sixteen downstream prompts.

The wall-time saving decomposes the same way. Both runs end when their pre-gather barrier releases, so comparing the two critical paths is exact in total. In the baseline the last arrival was rank 5: term extraction to 21.53 s, the glossary and register `allreduce`s to 30.27 s, translation to 98.79 s, halo to 100.50 s, revision to 166.52 s. Here the last arrival was rank 4: translation to 45.54 s, halo to 50.75 s, revision to 94.26 s. Stage by stage the difference is −21.53 s (extraction), −8.74 s (the two `allreduce`s), −22.98 s (translation), +3.50 s (halo), −22.51 s (revision), totalling −72.26 s against a measured wall difference of 72.23 s. The two `allreduce` invocations are 12% of what removing them saved and the extraction calls they consumed are another 30%, while translation and revision running on shorter prompts account for 63% and the lengthened halo wait gives 5% back.

That last step is an inference and needs its support stated. Fitting latency against prompt and output length over all 56 real agent calls in the three p8 arms driven by real agents gives latency $\approx 3.95 + 0.0116\,p + 0.0550\,o$ seconds, with $R^2 = 0.42$; the output coefficient is consistent with the fabric's own calibration ($\beta = 0.0559$ s per token). The prompt coefficient applied to the roughly 1,300-token glossary block predicts about 15 s of added latency per call, which brackets the observed per-call means: translation 63.90 s in the baseline against 45.89 s here, revision 47.70 s against 36.20 s. The regression is a consistency check rather than independent evidence, for the reason given in the threats section.

One correction to the headline comparison, because the coordination-share figures are not comparable as printed. `collective_wall_s` sums each invocation's longest per-rank wait over every invocation, including composed ones, so the baseline's two `allreduce` rows (10.238 and 0.748 s) are charged alongside the `reduce` and `bcast` rows they are made of. This run has no composed collectives, so its 19.001 s and 20.1% stand; the baseline's 59.474 s should be 48.49 s, a 29.1% share rather than 35.66%. A cleaner comparison uses rank-seconds, which are additive by construction: summing

each collective's per-rank blocking time gives 76.21 rank-seconds here out of the $8 \times 94.554 = 756.4$ available, or 10.1%, against 219.25 out of 1334.3, or 16.4%, in the baseline. Whichever of the two measures is used, removing the glossary cut coordination by about a third in relative terms, not by the 44% the printed figures imply.

The other half of the trade, which did not appear

The consistency cost is a quality measurement, not a trace measurement, and the result file exists: `results/translation/real-tr-p8-nogloss-halo-union-reduce_bcast-w600-u8.json` records `consistency` = 1.0 over 6 shared entities with 0 divergent, `coverage` = 1.0, `rendering_verified` = 1.0 over 22 reported renderings, and `paragraph_fidelity` = 1.0. So does the baseline. So does `nohalo`. So does `semanticgloss`. The metric that the entire glossary apparatus exists to move reads 1.000 in all four arms of the ladder.

Because a saturated metric proves little on its own, I checked the assembled text directly. Sections 0, 3 and 5 are byte-identical between this run and the baseline; sections 1, 2, 4, 6 and 7 match at character similarities of 0.982, 0.979, 0.999, 0.998 and 0.996. Counting occurrences of the baseline glossary's rendering for each of the ten pre-registered canonical entities in each of the eight section files gives identical counts in both runs, section for section. The surviving differences are lexical: the two runs choose different Chinese words for the anonymous note that opens section 2 and for Holmes's "method", and not one difference touches a canonical entity. In one instance the glossary arm is the worse of the two, rendering "seven and a half pounds" of body weight with the word for pounds sterling where this arm correctly uses the unit of weight.

The honest statement is therefore that this trace and its result file cannot establish a consistency cost for removing the glossary, because no arm of this ablation exhibits one. That is a fact about the benchmark, not a licence to conclude that collective agreement is unnecessary. Four reasons the metric sits at its ceiling, in decreasing order of how confident I am in them. Only 6 of the 10 pre-registered entities were rendered by two or more units, so `consistency` is a six-item test. The corpus is *A Scandal in Bohemia*, whose Chinese renderings are canonical and already in the model's prior — the glossary is largely telling the agents what they would have written anyway. The halo exchange survives in this arm and reported 27 boundary changes across the eight ranks against the baseline's 30, so a second reconciliation channel stayed open, although the one-paragraph boundaries it carried were 5 to 86 tokens and its repair capacity is correspondingly small. And the agreed glossary in the baseline is 206 terms with four conflicting proposals, dominated by Project Gutenberg front matter: unit 0 is the table of contents, so story titles such as *The Adventure of the Speckled Band* enter the glossary and are then broadcast to seven ranks whose text never mentions them.

The flagged findings

The `halo_exchange` accounting warning is a genuine AgentMPI defect and it has already been fixed. The current `src/agentmpi/topology.py` emits `algorithm`, `messages_sent`, `tokens_sent`, `rounds` and `wall_s` from `halo_exchange`; the payload in this trace carries only `{label, left, right}`, which is why the analysis reads algorithm `?`, zero messages and zero wall against fourteen messages the fabric logged under the `_ampi:halo` tag. The fix is commit `ee21d79`, "Report messages and tokens from the neighbourhood collectives"; this run's provenance commit is `b0b652d`, an ancestor of it. So the finding is expected for a trace of this vintage rather than a live bug — but it has

a consequence for every number in this document that derives from collective time. The halo contributes 0.00 s and 0 tokens to the 19.001 s collective wall, while the log shows its fourteen messages spanning 44.28 to 51.71 s with 7.43 s of skew and 22.97 rank-seconds of waiting. The 20.1% coordination share is a floor.

The stray-rank warning is expected. Ranks 8 and 14 emit two `rank.init` events each and nothing else: no messages, no agent calls, no finalize. The same two identifiers appear in every sibling run, which makes them an artefact of the shared worker pool registering against the job rather than anything the protocol did. They are worth noticing only because they inflate `n_ranks_seen` to 10 and contribute 2 of the 24 reattachments, so the per-rank tables and the reattachment count are not quite about the same population.

The reattachment finding is the one that answers the lifecycle question directly, and it is working as designed. Six ranks reached incarnation 4 and ranks 4 and 5 reached 3, for 24 reattachments against the baseline's 31. The pattern is one incarnation per agent phase: incarnation 1 is the harness's broker-side rank, then a fresh worker session claims each agent call, then a final session carries the rank into the barrier. The baseline has three agent phases and reaches incarnation 5; this run has two and reaches 4. Removing a collective did not change the reattachment mechanism, only how many times it fired — which is the answer one wants, since the mechanism should be a property of the runtime rather than of the harness's communication pattern. That rank identity genuinely survived the swap is visible in one message: mid 12, a 5-token left boundary, was sent by rank 2 at 44.04 s while rank 1 was running incarnation 2; rank 1 was replaced by a new worker session at 46.68 s; the message was delivered to that new session at 47.74 s, 3.705 s after it was sent. The mailbox outlived the agent that was waiting on it. Comparing each receiver's incarnation at send time and at receive time, 2 of the 28 messages cross a boundary, the other being a boot-ordering artefact at $t = 0.02$ s.

The cost-model agreement is real but thin, and Figure 5 shows how thin: it has three rows, and the halo is not one of them. `scatter` and `gather` each sent 7 messages in 3 rounds, exactly what the binomial closed form predicts for $p = 8$, and the barrier sent 0 as predicted. Two of the three passing checks are therefore the same closed form applied twice, and the barrier's tick is awarded on messages alone — the collectives table records it logging 0 rounds against a predicted 2, a disagreement the verdict column does not key on.

Two defects the analysis did not flag

The headline table says 8,161 tokens deferred. The job's own record in the same trace (`job.finish`) says 24,407. The difference is not rounding. `RankCost.tokens_deferred` is incremented on the send side for every rendezvous message and again on the receive side for every message not admitted into context, so every rendezvous transfer is counted twice and every unadmitted eager receive is counted once under a name that says rendezvous. Summing the log confirms the identity exactly: rendezvous send tokens 8,161 plus unadmitted receive tokens 16,246 equals 24,407, and the same holds in all three siblings ($8{,}273 + 31{,}603 = 39{,}876$ for `full`, $8{,}083 + 31{,}054 = 39{,}137$ for `nohalo`, $8{,}893 + 31{,}906 = 40{,}799$ for `semanticgloss`). The analysis pipeline's summariser counts only the send side and is the defensible number; the per-rank cost account, and the `per_rank_stats` block of every result JSON that copies it, is not. This one is live.

The second is the coordination-share arithmetic noted above, and it is a defect in the analysis layer rather than in the protocol. Summing each invocation's maximum per-rank wait over all invocations double-counts composed collectives and also adds intervals that overlapped in real time,

so the result is not a share of anything and is not bounded by wall time. `real-tr-p8-nohalo` is the reductio: 173.077 s of collective wall against a run that lasted 125.948 s, a printed coordination share of 137%, produced by charging the `glossary allreduce` (55.020 s) alongside the `reduce` (53.269 s) and `bcast` (55.018 s) it consists of. The analysis already carries a `composed_of` relation and uses it to avoid double-counting messages, so the first half is a one-line fix; the second half needs a denominator that the numerator can actually be a fraction of, which is what the rank-seconds form above supplies. The consequence for this document is that coordination shares must be compared within a definition and never across the two.

What this arm contributes to the ladder

Placed against its siblings, this is the cheap end: 204 events, 94.55 s and $0.1355, against `nohalo` at 263 events, 125.95 s and $0.1629, `full` at 347 events, 166.78 s and $0.2599, and `semanticgloss` at 398 events, 1152.90 s and $0.3536. The comparison with `nohalo` is the sharpest, because the two arms make the same number of agent calls — 16 each, eight translations plus eight of something else — and differ in which coupling they keep. Dropping the glossary is worth 31.4 s and 15,155 input tokens more than dropping the halo, which is what one should expect: the halo is fourteen messages and one revision phase, whereas the glossary is a full agent phase, two `allreduce`s, and a tax on every prompt that follows. And `semanticgloss`, which keeps the collective but swaps the exact `UNION` operator for a model-evaluated merge, costs 12.2× this run's wall time and still lands on `consistency` = 1.0 with all eight ranks holding the same glossary digest. Across the ladder the coordination arms differ by an order of magnitude in price and not at all in measured quality. Read as a whole, the four arms currently establish the cost curve of collective agreement and nothing about its value.

8 Threats to this reading

The wall-time and token deltas rest on one run per arm. Within this run, agent latency has p50 43.02 s, p95 50.52 s and max 51.52 s across sixteen calls, so per-call variance is visible; the run-to-run component is entirely unmeasured. The two runs being compared were recorded 94 seconds apart (result timestamps 06:54:10Z and 06:55:44Z), which is the best available protection against drifting broker load but is not a controlled comparison, and a shared worker pool can be busier at one moment than another for reasons no trace here records.

The latency regression is partly circular. Its prompt coefficient is fit across runs whose prompt lengths differ mainly because of the very glossary block whose effect I am using the coefficient to explain, so it cannot separate "longer prompts are slower" from "the run with glossaries happened to be slower". The stage-by-stage critical-path decomposition does not depend on the regression — it is arithmetic on timestamps — but the attribution of the 45.5 s residual to prompt length rather than to something else about that run does.

The critical-path decomposition uses a different rank in each run: rank 5 was the baseline's last arrival and rank 4 is this run's. The total is exact because both paths end at their barrier release, but the split between the translation and revision stages would shift by several seconds if a different rank had straggled, and with only eight samples per phase the per-stage figures should be read as approximate.

The strongest threat is to the negative quality result. `consistency` is computed from the agents'

self-reported `renderings` maps: 22 entries over 6 entities that recur in more than one section. `rendering_verified` = 1.0 closes the obvious hole — an agent claiming a rendering it did not write — and `coverage` = 1.0 confirms every entity the harness knew to be present was reported, so the metric is honestly constructed. It is simply a six-item test on a corpus where the six items have canonical translations. A negative result on it is weak evidence about glossaries and strong evidence about the benchmark.

Relatedly, three of eight sections came back byte-identical from two runs whose prompts differed by more than a thousand tokens and which each passed through two independent model calls. That points to near-deterministic decoding. If it is, then this ablation compares two nearly-deterministic samples and is structurally unable to observe the variance-driven divergence that a shared glossary exists to suppress — the failure mode is real but this experimental design cannot sample it.

This arm is not an isolation of "no coordination". The halo exchange survives, and the revision prompt still instructs each rank to fix terminology that disagrees with the abutting text; 27 boundary changes were reported. Some part of the unchanged consistency may be that channel doing the glossary's job at one-paragraph range, and this run cannot distinguish that from the agents never having disagreed. The arm that would distinguish them — no glossary and no halo — is not in the ladder.

Two properties of the artefacts limit what can be read out of them. The trace predates commit `ee21d79`, so its collective timings are not comparable with any trace recorded after that fix, and every coordination figure here is a lower bound. And context accounting is entirely inert: all 28 receives were unadmitted, context high-water is 0 on every rank, and the empty right panel of Figure 3 means only that this harness passes `admit=False` everywhere — nothing about context pressure, admission or eviction can be inferred from this run.

Finally, the executor was real. The configuration records `executor: broker`, and the log shows eight broker incarnations and twenty-six worker incarnations carrying sixteen model calls that took 28.5 to 51.5 s apiece. This is a statement about agent behaviour and not merely about the protocol, which is what makes the null quality result worth reporting — but it is one model, one language pair, and one 5,109-word book, and the conclusion that a glossary is redundant would not survive a corpus whose proper nouns the model has never seen.